

NEXA — Plano de Setup Técnico (v1)

1. Objetivo

Definir a estrutura inicial do projeto para permitir o início da implementação com segurança, consistência e visão de escala.

Este documento cobre:

- organização inicial do projeto
 - estrutura de pastas
 - base técnica recomendada
 - setup do banco
 - sequência de implementação inicial
 - diretrizes para web e evolução futura para mobile
-

2. Diretriz de Estrutura

O novo projeto deve ser iniciado separado da base atual.

A base atual permanece como:

- referência funcional
- fonte de aprendizado
- ambiente legado temporário

O novo projeto será a base oficial do NEXA.

3. Stack Recomendada

3.1 Frontend

- React
- TypeScript
- Vite
- React Router

3.2 Backend e dados

- Supabase Auth
- Supabase PostgreSQL
- Supabase Storage

3.3 Deploy

- Vercel

3.4 Motivo da escolha

Essa stack permite:

- velocidade de desenvolvimento
- boa integração com frontend
- autenticação pronta
- banco relacional consistente
- evolução futura com menor atrito

4. Estrutura Inicial de Pastas

Sugestão de estrutura base:

src/
app/
 router/
 providers/
 layout/

modules/
 auth/
 dashboard/
 negotiations/
 units/
 developments/
 clients/
 brokers/
 brokerages/
 settings/

domain/
 entities/
 enums/
 rules/
 services/

infrastructure/
 supabase/
 repositories/
 mappers/

shared/
 components/
 ui/
 hooks/
 utils/
 types/
 constants/

styles/

5. Critério de Separação de Camadas

5.1 app/

Responsável pela composição geral do sistema:

- roteamento
- providers globais
- layout principal

5.2 modules/

Cada módulo funcional da aplicação:

- tela
- componentes locais
- hooks locais
- lógica de apresentação

5.3 domain/

Coração do sistema:

- entidades do negócio
- regras
- enums de estados
- serviços de domínio

5.4 infrastructure/

Integração com mundo externo:

- Supabase
- repositories
- mapeamento de dados
- persistência

5.5 shared/

Elementos reutilizáveis:

- UI
 - hooks genéricos
 - utilitários
 - constantes
 - tipos comuns
-

6. Entidades que Devem Existir Desde o Início

No setup inicial, já devem existir definições de domínio para:

- Account
- Profile
- Development
- Unit
- Client
- Broker
- Brokerage
- Negotiation
- Proposal
- Reservation
- Sale

Mesmo que nem todas tenham implementação completa na primeira semana, a estrutura deve nascer preparada.

7. Banco de Dados — Primeira Onda

As primeiras tabelas a serem criadas devem ser:

1. `accounts`
2. `profiles`
3. `user_account_access`
4. `developments`
5. `units`
6. `clients`
7. `brokerages`
8. `brokers`
9. `negotiations`

Essas tabelas formam a fundação mínima do sistema.

8. Banco de Dados — Segunda Onda

Após a fundação, entram:

10. `proposals`
11. `proposal_assets`
12. `reservation_requests`
13. `reservations`
14. `sales`
15. `unit_queue`

16. `unit_status_history`

17. `activity_logs`

18. `account_settings`

19. `development_settings`

20. `materials`

9. RLS e Autorização

O sistema deve nascer já pensando em segregação real por conta.

Princípio

Toda linha de dado comercial precisa estar vinculada a `account_id`.

Diretriz

As policies devem partir do acesso do usuário ao workspace.

Padrão preferencial:

```
SELECT role FROM profiles WHERE profiles.user_id = auth.uid()
```

Evitar políticas excessivamente complexas no início.

10. Setup Inicial do Frontend

O frontend deve começar com:

10.1 Providers globais

- AuthProvider
- AccountContext
- DevelopmentContext

- ThemeProvider

10.2 Fluxo inicial

1. login
 2. identificação de contas acessíveis
 3. seleção de conta
 4. seleção de empreendimento
 5. entrada no sistema
-

11. Primeiras Telas a Construir

Ordem recomendada:

Fase inicial

1. Login
 2. Seleção de conta
 3. Seleção de empreendimento
 4. Layout base com sidebar
 5. Dashboard inicial
 6. Cadastro de empreendimentos
 7. Cadastro de unidades
 8. Cadastro de clientes
 9. Cadastro de corretores
 10. Cadastro de negociações
-

12. Primeiros Módulos Operacionais

Após o setup visual e estrutural:

Prioridade 1

- developments
- units
- clients
- brokers
- negotiations

Prioridade 2

- proposals
- reservations
- sales

Prioridade 3

- queue
- materials
- analytics refinado
- configurações avançadas

13. Estados que Devem Nascer Como Enum

Para evitar caos futuro, estados devem nascer centralizados em enums.

Exemplo de diretório:

src/domain/enums/

Enums obrigatórios:

- UnitStatus
- NegotiationStatus
- ProposalStatus
- ReservationStatus
- SaleStatus
- UserRole

14. Regras que Não Devem Ficar em Tela

As seguintes regras devem ir para domínio/serviço, nunca para componente React:

- mudança de status da unidade
- validação de reserva
- entrada em fila
- promoção na fila
- vínculo entre negociação e proposta
- conclusão de venda

15. Integração com Mobile

Mesmo que o app mobile venha depois, o projeto web já deve nascer preparado para isso.

Diretrizes

- evitar regra enterrada em componente
- centralizar domínio

- manter contratos claros entre UI e dados
- usar estrutura reutilizável

Implicação

Se isso for respeitado, depois será possível criar:

- app mobile em React Native/Expo
 - consumindo a mesma lógica de backend e modelo de dados
-

16. Convenções Iniciais

Código

- TypeScript estrito
- sem lógica de negócio em JSX
- sem duplicação de estados de domínio

CSS

- tokens centralizados
- manter consistência com a identidade visual definida
- evitar espalhar variáveis por arquivos locais

Nomenclatura

- nomes em inglês para entidades técnicas
 - nomes claros e previsíveis
 - evitar abreviações confusas
-

17. Setup Operacional no VS Code

Ao iniciar o projeto:

1. criar repositório novo
 2. iniciar projeto Vite com React + TS
 3. configurar ESLint e Prettier
 4. configurar Supabase
 5. criar estrutura de pastas
 6. criar enums e entidades base
 7. criar tabelas iniciais
 8. subir autenticação e fluxo de conta/empreendimento
-

18. Sequência Recomendada de Execução Inicial

Semana 1

- repositório novo
- base React + TS + Vite
- Supabase configurado
- estrutura de pastas criada
- tabelas base criadas

Semana 2

- login
- account selection
- development selection

- layout principal

Semana 3

- cadastros base
- units
- clients
- brokers

Semana 4

- negotiations
- vínculo inicial com units
- fluxo mínimo operacional

19. Critério de Validação do Setup

O setup só estará corretamente concluído quando:

- usuário conseguir autenticar
- sistema reconhecer a conta
- sistema exigir empreendimento ativo
- dados estiverem segregados por conta
- unidades estiverem persistidas como entidade real
- negociação já não depender do simulador legado

20. Síntese Final

O setup técnico do NEXA precisa ser construído para sustentar o produto que foi definido, e não apenas para reproduzir o sistema atual com nova interface.

A qualidade do projeto dependerá da disciplina em respeitar:

- separação de camadas
- centralização das regras
- multi-tenant desde o início
- unidade e negociação como núcleo do sistema